

Prognozowanie średniej dobowej temperatury powietrza z wykorzystaniem metod uczenia maszynowego

Tomasz Jarko, tomasz.jarko@gmail.com

Uniwersytet Przyrodniczy w Poznaniu, 60-637 Poznań

1. Wstęp:

Jednym z najistotniejszych zastosowań metod analizy danych jest obecnie prognozowanie pogody. Warunki atmosferyczne takie jak wielkość opadów, temperatura minimalna oraz temperatura maksymalna mają duże znaczenie dla niektórych sektorów gospodarki, takich jak transport, energetyka, gospodarka komunalna oraz przede wszystkim rolnictwo. Coraz większa ilość danych gromadzonych przez liczne czujniki oraz stacje meteorologiczne pozwala nam na dokładniejsze interpretowanie i przewidywanie pogody. Pojawia się również możliwość wykorzystania nowych narzędzi, metod sztucznej inteligencji do wspomagania procesów prognozowania warunków pogodowych.

Tradycyjne modele prognostyczne wykorzystują zjawiska fizyczne w celu opisanie zależności zachodzących w atmosferze. W ostatnich latach coraz większe zainteresowanie w dziedzinie prognozowania pogody budzą metody sztucznej inteligencji takie jak uczenie maszynowe (machine learning) oraz głębokie uczenie (deep learning), które umożliwiają wykrywanie zależności zachodzących w dużych zbiorach danych bez konieczności opisywania wszystkich procesów fizycznych. Algorytmy takie jak regresja liniowa, drzewa decyzyjne czy sieci neuronowe znajdują realne zastosowania w analizie danych meteorologicznych oraz wspomaganie prognozowania pogody przez naukowców.

Według dostępnej literatury, metody uczenia maszynowego mogą osiągnąć wysoką skuteczność w zadaniach związanych z prognozowaniem warunków atmosferycznych w szczególności przy wykorzystaniu dużych zbiorów składających się z wieloletnich danych historycznych.

W niniejszym projekcie wykorzystano połączenie dostępnych danych pomiarowo-obszaryjnych, meteorologicznych i atmosferycznych pochodzących ze strony API Instytutu

Meteorologii i Gospodarki Wodnej. Zakres danych, jaki obejmuje przygotowana baza danych wynosi 24 lata, od roku 2001 do 2025 roku. Dane obejmują kolejne dni pomiarowe, co umożliwia analizę zmian warunków atmosferycznych w czasie. Analiza została przeprowadzona na podstawie danych z jednej stacji pogodowej wybranej dla każdego z szesnastu województw w Polsce.

Prognozowanie temperatury powietrza stanowi klasyczny problem regresyjny, który jest często wykorzystywany do oceny skuteczności metod uczenia maszynowego. Z uwagi na sezonowość zjawisk atmosferycznych oraz występowanie długoterminowych trendów klimatycznych, dane meteorologiczne są interesującym źródłem informacji do budowy modeli predykcyjnych. Analiza takich danych pozwala nie tylko na ocenę dokładności wybranych algorytmów, ale również na identyfikację czynników mających największy wpływ na zmiany temperatury w czasie.

Celem projektu jest zastosowanie wybranych metod uczenia maszynowego do prognozowania średniej temperatury dobowej powietrza w wybranych województwach Polski na podstawie dobowych, historycznych danych meteorologicznych. W ramach prac zostanie przeprowadzone zebranie i wstępne przetwarzanie (pre-processing) danych, wybór odpowiednich cech wejściowych, budowa modelu predykcyjnego, stworzenie strony do wizualizacji danych oraz ocena skuteczności modelu na podstawie wybranych miar jakości. Uzyskane wyniki pozwolą określić przydatność praktycznego zastosowania metod sztucznej inteligencji w analizie i prognozowaniu danych meteorologicznych.

2. Dane i źródło danych:

Dane pochodzą z Instytutu Meteorologii i Gospodarki Wodnej:

https://danepubliczne.imgw.pl/data/dane_pomiarowo_obserwacyjne/dane_meteorologiczne/.

Dane obejmują 25 lat. Są to dane dobowe, obejmujące między innymi temperaturę minimalną, maksymalną i średnią.

Skrypt pobierający i zapisujący pliki .zip z danymi:

```
from pathlib import Path
import requests

output_dir = Path("zip_files")
output_dir.mkdir(exist_ok=True)

year = 2001
month = "01"

for _ in range(25 * 12):
    main_url = f"https://danepubliczne.imgw.pl/data/dane_pomiarowo_obserwacyjne/dane_meteorologiczne/dobowe/klimat/{year}/{year}_{month}_k.zip"

    response = requests.get(main_url)
    if response.status_code == 200:
        filename = output_dir / f"{year}_{month}_k.zip"
        filename.write_bytes(response.content)
        print(f"Zapisano: {filename}")
    else:
        print(f"Nie udało się pobrać: {main_url} ({response.status_code})")

    month = f"{int(month) + 1:02d}"
    if month == "13":
        year += 1
        month = "01"
```

Po uruchomieniu skryptu, otrzymujemy 300 plików zip zawierających dane historyczne, na których później będą wykonywane dalsze działania.

3. Przygotowanie danych:

W poniższej sekcji omówię proces przygotowania danych (ang. *data preprocessing*), którego celem było przekształcenie surowych danych meteorologicznych do postaci umożliwiającej ich dalszą analizę oraz wykorzystanie w modelach uczenia maszynowego. Dane pozyskane z zasobów IMGW wymagały wcześniejszego oczyszczenia, ujednolicenia oraz połączenia w jeden spójny zbiór danych.

Proces przygotowania danych składał się z następujących etapów:

3.1. Rozpakowanie archiwów ZIP:

Dane meteorologiczne zostały pobrane w postaci skompresowanych archiwów ZIP. W pierwszym kroku wszystkie archiwa zostały rozpakowane do katalogu roboczego. Podczas tego procesu uwzględniono obsługę błędów umożliwiającą wykrywanie uszkodzonych plików archiwów oraz pomijanie ich w dalszym przetwarzaniu.

```

import zipfile
import glob
from zipfile import BadZipFile

def unpack_zip(zip_path, extract_to):
    with zipfile.ZipFile(zip_path, 'r') as zip_ref:
        zip_ref.extractall(extract_to)

# Etap 1
# Rozpakowanie wszystkich plików zip znajdujących się w katalogu 'zip_files' do katalogu 'tmp'
for zip_file in glob.glob('zip_files/*.zip'):
    try:
        unpack_zip(zip_file, 'tmp/')
        print(f"Plik {zip_file} został rozpakowany")
    except BadZipFile:
        print(f"Plik {zip_file} jest uszkodzony")

```

3.2. Selekcja odpowiednich plików:

Po rozpakowaniu danych wybrano odpowiednie pliki do dalszej analizy. Usunięto pliki zawierające w nazwie literę „t”, które odpowiadały danym terminowym (pomiarom wykonywanym w określonych godzinach). W projekcie wykorzystano wyłącznie dane dobowe, dlatego pliki te nie były potrzebne do dalszej analizy.

```

# Etap 2
# Usuwanie plików zawierających "t" w nazwie, czyli plików z danymi terminowymi
import os
for filename in os.listdir('tmp'):
    if 't' in filename:
        os.remove(os.path.join('tmp', filename))
        print(f"Plik {filename} został usunięty")

```

3.3. Wczytywanie i scalanie danych:

Kolejnym etapem było wczytanie wszystkich plików CSV do środowiska Python z wykorzystaniem biblioteki Pandas. Ze względu na możliwość występowania różnych kodowań znaków zastosowano mechanizm automatycznego testowania kilku popularnych kodowań (UTF-8, CP1250, Latin-1 oraz CP1252).

Podczas importu przypisano nazwy kolumn zgodnie z dokumentacją IMGW, obejmujące między innymi:

- nazwę stacji meteorologicznej,
- rok, miesiąc i dzień pomiaru,
- temperaturę maksymalną,
- temperaturę minimalną,
- średnią temperaturę dobową

Po poprawnym wczytaniu wszystkich plików zostały one połączone w jeden zbiór danych.

```

# =====
# FUNKCJA WCZYTAJĄCA JEDEN PLIK
# =====
def read_single_csv(file_path, columns):
    if os.path.getsize(file_path) == 0:
        print(f"Plik pusty, pomijam: {file_path}")
        return None

    last_error = None

    for enc in encodings_to_try:
        try:
            df = pd.read_csv(
                file_path,
                sep=";",
                header=None,
                names=columns,
                encoding=enc,
                engine="python",
                skipinitialspace=True,
                na_values=["", " ", "NA", "N/A", "nan"],
                keep_default_na=True,
                on_bad_lines="skip"
            )

            # Jesli dataframe pusty po wczytaniu
            if df.empty:
                print(f"Plik po wczytaniu pusty, pomijam: {file_path}")
                return None

            # Czyszczenie spacji w kolumnach tekstowych
            for col in df.select_dtypes(include="object").columns:
                df[col] = (
                    df[col]
                    .astype(str)
                    .str.replace(r"\s+", " ", regex=True)
                    .str.strip()
                )

            # Zamiana pustych stringow na NaN
            df = df.replace(r"^\s*$", np.nan, regex=True)

            # Usuniecie wierszy calkowicie pustych
            df = df.dropna(how="all")

            # Jesli kolumna POST istnieje, wyczyszc ja jeszcze raz
            if "POST" in df.columns:
                df["POST"] = (
                    df["POST"]
                    .astype(str)
                    .str.replace(r"\s+", " ", regex=True)
                    .str.strip()
                )

```

Powżej znajduje się fragment kodu wykorzystywanego w kroku 3.3.

3.4. Czyszczenie danych:

W celu poprawy jakości danych wykonano szereg operacji czyszczących:

- usunięcie nadmiarowych spacji i znaków białych z nazw stacji meteorologicznych,
- zastąpienie pustych wartości oraz niepoprawnych wpisów wartością *NaN*,
- usunięcie całkowicie pustych rekordów,
- standaryzację zapisów tekstowych,
- konwersję wartości numerycznych do odpowiednich typów danych.

```

# Usuniecie białych znakow z nazw kolumn
merged_df.columns = [str(col).strip() for col in merged_df.columns]

```

3.5. Wynik procesu przygotowania danych:

Efektom przeprowadzonych operacji był jeden ujednolicony zbiór danych zapisany w pliku *merged_data.csv*, zawierający dane meteorologiczne z wielu stacji pomiarowych oraz wielu lat obserwacji. Zbiór ten stanowił podstawę do dalszej analizy eksploracyjnej danych oraz budowy modeli prognozowania temperatury. Poniżej znajduje się przykładowy wgląd do przygotowanego pliku:

```
metody_si > projekt > dane > merged_data.csv
1  NSP, POST, ROK, MC, DZ, TMAX, WTMX, TMIN, WTMN, STD, WSTD, TMNG, WTMNG, SMDB, WSMD, ROOP, PKSN, WPKSN
2  249180010, PSZCZYNA, 2001, 1, 1, -1.3, -9.6, -5.7, -11.0, 0.0, 9.0, 13.0,
3  249180010, PSZCZYNA, 2001, 1, 2, 3.3, -12.0, -2.7, -13.2, 0.0, 9.0, 11.0,
4  249180010, PSZCZYNA, 2001, 1, 3, 1.5, -5.7, -1.5, -6.8, 0.0, 9.0, 10.0,
5  249180010, PSZCZYNA, 2001, 1, 4, 6.5, -1.1, 1.9, -4.8, 0.0, 9.0, 8.0,
6  249180010, PSZCZYNA, 2001, 1, 5, 6.5, -2.3, 2.8, -6.5, 0.3, 7.0,
7  249180010, PSZCZYNA, 2001, 1, 6, 8.0, 3.1, 5.3, -0.4, 2.0, 3.0,
8  249180010, PSZCZYNA, 2001, 1, 7, 6.0, 2.0, 3.6, 1.7, 4.7, 1.0,
9  249180010, PSZCZYNA, 2001, 1, 8, 4.1, 1.3, 2.7, 2.2, 23.4, 0.0, 9.0
10 249180010, PSZCZYNA, 2001, 1, 9, 2.8, 0.0, 1.4, -0.9, 3.4, 0.0, 9.0
11 249180010, PSZCZYNA, 2001, 1, 10, 4.0, -1.4, 0.7, -2.8, 2.0, 0.0, 9.0
12 249180010, PSZCZYNA, 2001, 1, 11, 0.6, -4.8, -1.8, -6.3, 4.8, 4.0,
13 249180010, PSZCZYNA, 2001, 1, 12, -0.4, -6.4, -3.0, -6.0, 0.1, 3.0,
14 249180010, PSZCZYNA, 2001, 1, 13, -1.8, -5.6, -3.5, -7.3, 0.0, 3.0,
15 249180010, PSZCZYNA, 2001, 1, 14, -1.5, -8.6, -4.7, -9.7, 0.0, 9.0, 3.0,
16 249180010, PSZCZYNA, 2001, 1, 15, -1.5, -9.1, -5.0, -10.7, 0.0, 9.0, 3.0,
17 249180010, PSZCZYNA, 2001, 1, 16, -2.5, -10.2, -6.7, -10.2, 0.0, 9.0, 3.0,
18 249180010, PSZCZYNA, 2001, 1, 17, -4.4, -9.1, -6.7, -10.2, 0.0, 3.0,
19 249180010, PSZCZYNA, 2001, 1, 18, -4.5, -10.3, -7.5, -10.2, 0.0, 3.0,
20 249180010, PSZCZYNA, 2001, 1, 19, -4.9, -9.1, -6.8, -9.6, 0.0, 9.0, 3.0,
21 249180010, PSZCZYNA, 2001, 1, 20, 0.0, -8.2, -3.2, -9.4, 0.0, 9.0, 3.0,
22 249180010, PSZCZYNA, 2001, 1, 21, 0.0, -7.2, -4.7, -8.7, 0.0, 9.0, 3.0,
23 249180010, PSZCZYNA, 2001, 1, 22, -4.0, -7.7, -5.4, -8.7, 0.0, 3.0,
24 249180010, PSZCZYNA, 2001, 1, 23, 0.5, -7.2, -2.7, -8.9, 1.1, 3.0,
25 249180010, PSZCZYNA, 2001, 1, 24, 6.3, -8.1, -0.9, -9.3, 8.8, 4.0,
```

Całość prowadzonych prac znajduje się w repozytorium **GitHub** dostępny pod adresem: https://github.com/Tomson601/upp_csde/tree/main/metody_si/projekt